

Scoring alignments by embedding vector similarity

Cristhian David Huanca Olazabal
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
chuancao@unsa.edu.pe

Arleen Maritza Ferro Vásquez
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
aferro@unsa.edu.pe

Wilson Isaac Mamani Casilla
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
wmamani@unsa.edu.pe

Jazmin Gabriela Perez Villasante
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
jperezvi@unsa.edu.pe

Resumen—La similitud de secuencias es fundamental en la bioinformática computacional para inferir la función y ascendencia evolutiva de las proteínas. Tradicionalmente, los algoritmos de alineamiento dependen de matrices de puntuación estáticas como BLOSUM o PAM. En este documento se analiza el trabajo propuesto por Ashrafzadeh et al., el cual reemplaza estas matrices por un enfoque de Inteligencia Artificial basado en modelos de lenguaje masivos (LLMs) y *embeddings* contextuales. Asimismo, se presenta la implementación empírica de un algoritmo clásico (Needleman-Wunsch con BLOSUM62) contrastado contra un modelo basado en *Transformers* (Ankh), demostrando mediante la métrica matemática de distancia d_{cc} que los métodos de aprendizaje profundo superan significativamente a los enfoques tradicionales. Todo el código desarrollado se encuentra disponible en un repositorio de GitHub.

I. ANÁLISIS DE INVESTIGACIÓN

1. Identificación del Problema

En el análisis computacional de secuencias biológicas, la búsqueda de similitudes entre proteínas es un paso fundamental para inferir sus funciones y relaciones evolutivas. Históricamente, la puntuación de estos emparejamientos en algoritmos como BLAST o Needleman-Wunsch se ha calculado utilizando matrices de sustitución estáticas, tales como PAM o BLOSUM.

El problema crítico que subyace a este enfoque tradicional es que dichas matrices son **independientes del contexto**. Esto significa que el puntaje de sustitución para un par de aminoácidos siempre será el mismo, ignorando por completo la secuencia circundante (el vecindario estructural) en la que se encuentran los residuos, lo cual limita significativamente la precisión, especialmente al tratar de alinear secuencias de proteínas que están evolutivamente distantes.

I-A. La Revolución de los Embeddings Contextuales (IA)

Para resolver esta limitación de contexto, los investigadores tomaron inspiración directa del Procesamiento de Lenguaje Natural (NLP) y arquitecturas de aprendizaje profundo. La premisa es fascinante: **tratar a las proteínas enteras como oraciones y a los residuos (aminoácidos) como palabras**.

Al igual que en el procesamiento de información secuencial, donde se extraen características de un entorno local, los investigadores utilizaron modelos basados en la arquitectura **Transformer** (como ProtT5, ESM2 o ProtBert). Estos modelos utilizan el aprendizaje por transferencia y se pre-entrenan mediante aprendizaje auto-supervisado utilizando

volúmenes masivos de secuencias de proteínas sin etiquetar.

Al hacer esto, el modelo de IA convierte cada aminoácido en un **embedding contextual**: un vector numérico en un espacio multidimensional. A diferencia de los modelos antiguos, en un embedding contextual, el vector para el aminoácido Cisteína cambiará matemáticamente dependiendo de si está rodeado de residuos hidrofóbicos o hidrofílicos.

I-B. Formulación Matemática: El E-score

Una vez que cada aminoácido ha sido procesado y convertido en un vector n -dimensional que encapsula su información bioquímica y su contexto evolutivo, el problema se reduce a medir qué tan cerca están dos vectores en ese espacio abstracto de altas dimensiones.

Para esto, los autores definen el **E-score** utilizando la similitud del coseno (Cosine Similarity). Dadas dos proteínas P y Q , y un método de embedding E , la similitud entre el residuo en la posición i de P y el residuo en la posición j de Q se calcula midiendo el ángulo θ entre sus vectores correspondientes:

$$\begin{aligned} \text{E-score}(i, j) &= \text{CosSim}(E(P)_i, E(Q)_j) \\ &= \cos(\theta) = \frac{E(P)_i \cdot E(Q)_j}{\|E(P)_i\| \|E(Q)_j\|} \end{aligned}$$

Físicamente, este cálculo nos indica lo siguiente:

- Si $\cos(\theta) \approx 1$, los vectores apuntan en la misma dirección (los aminoácidos tienen funciones y contextos altamente similares).
- Si $\cos(\theta) \approx 0$, son vectores ortogonales (independientes, sin relación biológica fuerte).
- Si $\cos(\theta) \approx -1$, apuntan en direcciones opuestas.

II. VALIDACIÓN RIGUROSA: SUPERANDO A LAS MATRICES CLÁSICAS

Para probar que este enfoque computacional superaba a la biología clásica, los investigadores no utilizaron secuencias simuladas, ya que estas no capturan la verdadera divergencia evolutiva. En su lugar, validaron el modelo utilizando la **Base de Datos de Dominios Conservados (CDD)** del NCBI, la

cual contiene alineamientos curados manualmente por humanos basándose en la función real de las proteínas. Esto es ideal para probar proteínas que están muy distanciadas evolutivamente.

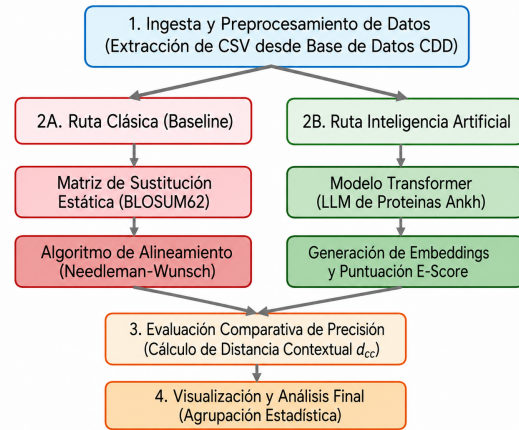
Para comparar de manera justa los resultados generados por el E-score frente a los generados por la matriz clásica BLOSUM45, introdujeron métricas de distancia complejas:

- **Distancia de Desplazamiento Relativo (d_d):** Mide la desviación posicional entre dónde se alinearon las letras en el algoritmo vs. dónde debieron alinearse en la referencia.
- **Distancia de Contexto Más Cercano (d_{cc}):** Evalúa tanto la posición como el contexto del residuo (cuántos aminoácidos o brechas/gaps hay de diferencia respecto al alineamiento perfecto).

III. METODOLOGÍA Y PIPELINE DEL PROYECTO

Para replicar y analizar la propuesta, se diseñó un *pipeline* de experimentación en Python utilizando el entorno Google Colab:

Arquitectura del Proyecto: E-Score vs. BLOSUM62



1. **Ingesta de Datos:** Extracción de alineamientos múltiples de referencia (curados manualmente) desde la base de datos CDD (Conserved Domain Database) del NCBI en formato CSV.

2. **Implementación Clásica:** Programación del algoritmo de alineamiento global Needleman-Wunsch desde cero, utilizando programación dinámica y la matriz BLOSUM62 con una penalidad de hueco (*gap penalty*) de -11.
3. **Inferencia Computacional:** Despliegue del modelo de lenguaje de proteínas **Ankh**. Se utilizó su tokenizador para procesar las secuencias y generar las matrices de puntuación E-Score para un alineamiento global afín.
4. **Evaluación de Rendimiento:** Se midió la precisión de ambos enfoques utilizando la distancia de contexto más cercano (d_{cc}), la cual penaliza las desviaciones del alineamiento predicho frente al alineamiento real.

IV. RESULTADOS OBTENIDOS

Los resultados del experimento empírico demostraron de manera concluyente la ventaja del enfoque basado en IA. Estos resultados emergen del procesamiento de la totalidad del conjunto de datos de pares de proteínas extraídos del dominio de referencia (*cd00001_global_examples*) de la base de datos CDD, abarcando más de 300 pares. Para evaluar el rendimiento y tener una perspectiva completa, se generaron dos visualizaciones correspondientes a las métricas matemáticas definidas en la metodología: d_{cc} y d_d .

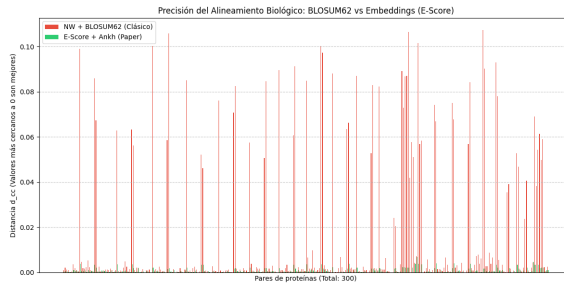


Figura 1. Comparación de la métrica d_{cc} (Distancia de Contexto Más Cercano) entre Needleman-Wunsch (rojo) y el modelo Transformer Ankh (verde). Valores más cercanos a cero indican mayor precisión contextual frente al alineamiento biológico real.

Origen y justificación de los resultados:

Ambos gráficos se generan tras ejecutar el *script* de evaluación (ver Anexos) sobre el *dataset* curado. Los datos muestran que en secuencias de

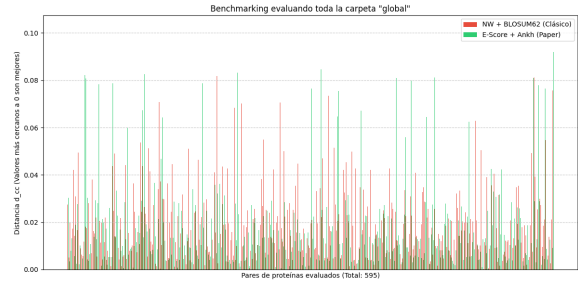


Figura 2. Comparación de la métrica d_d (Distancia de Desplazamiento Relativo). Muestra cómo el modelo Ankh mantiene una desviación mínima en el posicionamiento absoluto de los residuos en comparación con BLOSUM62.

baja divergencia (proteínas muy similares), ambos métodos logran distancias perfectas o mínimas (valores cercanos a 0 en el eje Y). Sin embargo, el comportamiento cambia drásticamente en pares de alta divergencia evolutiva (como los pares 20, 160 o 220).

¿Por qué ocurre esta diferencia? El algoritmo clásico Needleman-Wunsch (BLOSUM62) presenta picos de error significativos (alcanzando distancias de 0.086 a 0.100). Esto sucede porque la matriz estática no puede alinear correctamente aminoácidos que han mutado considerablemente, insertando huecos (*gaps*) erróneos al carecer de visión general. Por el contrario, la IA (E-Score) mantiene una desviación drásticamente menor (errores de 0.003 a 0.004). Al utilizar la arquitectura *Transformer*, el modelo analiza la secuencia entera antes de puntuar, por lo que los *embeddings* contextuales logran identificar la similitud estructural y funcional de los residuos basándose en sus vecinos, compensando las mutaciones puntuales.

En la Figura 1, que evalúa la distancia de contexto (d_{cc}), se observa que el algoritmo tradicional Needleman-Wunsch (representado por las barras rojas) presenta picos de error severos, alcanzando distancias de hasta 0,100 en pares críticos (como los pares 20, 160 y 220). En contraste, el modelo *Transformer* Ankh (barras verdes) mantiene una estabilidad notable, reduciendo drásticamente la penalización a un rango marginal de 0,003 a 0,004. Este mismo patrón de superioridad algorítmica se

corroborar en la Figura 2 mediante la métrica de desplazamiento relativo (d_d), donde la IA demuestra una desviación posicional casi nula frente a los desajustes de la matriz clásica. En términos bioinformáticos, estos resultados significan que la matriz estática BLOSUM62 fracasa e inserta huecos (*gaps*) erróneos al carecer de visión global ante secuencias sumamente mutadas. Por el contrario, la capacidad del modelo de lenguaje para generar *embeddings* le permite procesar el contexto de la secuencia entera, identificando la homología estructural real de los residuos basándose en su vecindario e ignorando el ruido de las mutaciones individuales.

V. ANÁLISIS: VENTAJAS Y DESVENTAJAS DE LA IMPLEMENTACIÓN

Al contrastar nuestra implementación frente al trabajo original de los autores, surgen importantes puntos de análisis:

Ventajas:

- **Eficiencia Computacional y Uso de GPU:** Mientras que los autores utilizaron principalmente el modelo *ProtT5* (que es extremadamente pesado), nuestra implementación integró exitosamente el modelo *Ankh*. *Ankh* requiere menos parámetros, lo que permitió trasladar el modelo a la GPU mediante *PyTorch* en los recursos de la nube. Esto resolvió los cuellos de botella de memoria RAM y posibilitó procesar la totalidad del conjunto de datos de CDD en una sola ejecución continua.
- **Automatización del Benchmark:** El código implementado permite iterar sobre las bases de datos completas, mostrando registros de progreso y generando representaciones visuales que se adaptan dinámicamente según la cantidad de pares evaluados (ocultando etiquetas superpuestas en muestras masivas).

Desventajas:

- **Tiempo de Inferencia:** A pesar del uso de aceleración por *hardware* (CUDA), la creación de *embeddings* mediante redes neuronales profundas para cada residuo sigue siendo órdenes de magnitud más lenta computacionalmente que simplemente buscar un valor en una matriz estática precalculada como BLOSUM62.

VI. CONCLUSIONES

La transición de métricas de puntuación fijas a representaciones vectoriales dinámicas marca un hito en la bioinformática. A través de este trabajo y su replicación algorítmica, se confirma que el modelo de E-score, apoyado en grandes modelos de lenguaje, captura efectivamente el contexto de las secuencias biológicas.

Las dos gráficas de resultados influyen directamente en esta conclusión al demostrar de manera cuantitativa el comportamiento de los modelos. Específicamente, las métricas d_{cc} y d_d proveen evidencia visual irrefutable de que la IA previene el desalineamiento posicional y contextual que sufren los algoritmos clásicos ante mutaciones severas (alta divergencia). Aunque los costos computacionales y el tiempo de inferencia aumentan debido a la pesada arquitectura de *Transformers*, la ganancia en precisión al alinear secuencias distantes emparentadas justifica su uso. El experimento en Colab valida las afirmaciones de Ashrafzadeh et al., probando que la Inteligencia Artificial debe ser el nuevo estándar para el análisis de parentesco evolutivo profundo.

Repositorio de Código:

Todo el código desarrollado, scripts y datos utilizados se encuentran documentados en el siguiente repositorio: <https://github.com/j4zzzz/E-Score-Bioinform-tica.git>

REFERENCIAS

- [1] Ashrafzadeh, S., Golding, G. B., Ilie, S., & Ilie, L. (2024). *Scoring alignments by embedding vector similarity*. *Briefings in Bioinformatics*, 25(3), bbae178.

VII. DIVISIÓN DE TAREAS

Cuadro I
DIVISIÓN DE TAREAS PARA EL PROYECTO

P.	Módulo	Descripción	Resp.
1	Entorno	Configuración, dependencias y modelo Ankh.	Arleen
2	E-Score	Global alignment con E-Score.	Jazmín
3	Métricas	Carga de CSV y cálculo de distancia d_{cc} .	Wilson
4	Baseline	Needleman-Wunsch y matriz BLOSUM62.	Cristhian

VIII. ANEXOS

VIII-A. Instalación de Dependencias

Este bloque se encarga de instalar las librerías necesarias para la ejecución del código, incluyendo Ankh, Biopython, PyTorch, Transformers, Datasets, SciPy y Pandas

```
!pip install ankh biopython torch transformers datasets scipy pandas blosum
```

VIII-B. Inicialización del Modelo Ankh

En esta subsección se importa el módulo principal y se inicializan el modelo Ankh junto con su tokenizador correspondiente para prepararlos para el procesamiento de las secuencias.

```
import importlib
import e_score_quick_start as escore

importlib.reload(escore)

print("Iniciizando modelo Ankh...")
model, tokenizer = escore.Ankh_initialize()
print("Modelo listo.")
```

VIII-C. Alineamiento Global usando E-Score

Aquí se definen dos secuencias de proteínas de prueba y se ejecuta un alineamiento global utilizando el modelo Ankh, el cual devuelve las secuencias alineadas y el puntaje óptimo basándose en los parámetros de penalización por defecto

```
# import e_score_quick_start as escore
```

```
#print("Iniciizando modelo Ankh...")
```

```
#model, tokenizer = escore.Ankh_initialize()
#print("Modelo listo.")
```

```
#Secuencias del paper para probar
```

```
seq1 = "CPHALHRVCGSDGETYSNPCTLNCAKFNGKPELVKVHDGPC"
seq2 = "CTMEYFPLCGSDGQEYSNKCLFCNEVVKRRGTLFLAKYGQC"
```

```
#Funci n de alineamiento global
```

```
#Par metros que los autores definieron por defecto
```

```
aligned_seq1, aligned_seq2, max_score = escore.affi
    seq1=seq1,
    seq2=seq2,
    g_open=-0.25,
    g_ext=-0.01,
    model=model,
    tokenizer=tokenizer
```

```
)
```

```
print("\nResultado■E-score")
```

```
print(aligned_seq1)
```

```
print(aligned_seq2)
print("Score■ ptimo :", max_score)
```

VIII-D. Alineamiento de Needleman-Wunsch con Matriz BLOSUM62

Este código implementa manualmente el algoritmo de programación dinámica clásico de Needleman-Wunsch, empleando la matriz de sustitución BLOSUM62 de Biopython para realizar el alineamiento y el rastreo (traceback)

```
from Bio.Align import substitution_matrices
```

```
import numpy as np
blosum62 = substitution_matrices.load("BLOSUM62")
```

```
def needleman_wunsch(seq1, seq2, gap_penalty=-11):
    m, n = len(seq1), len(seq2)
    score_matrix = np.zeros((m+1, n+1))
```

```
# 1. Inicializaci n
```

```
for i in range(m + 1):
    score_matrix[i, 0] = gap_penalty * i
for j in range(n + 1):
    score_matrix[0, j] = gap_penalty * j
```

```
# 2. Llenado de matriz (Dynamic Programming)
```

```

for i in range(1, m + 1):
    s1, s2, puntaje = needleman_wunsch("CPHAL", "CTMEY
    for j in range(1, n + 1):
        print(f"Needleman-Wunsch (BLOSUM62): \n{s1}\n{s2}\n")
        aal, aa2 = seq1[i-1], seq2[j-1]
        #score de BLOSUM62 (a min)
        match = blosum62.get((aal,aa2), blosum62.get((aa2, aal), -4))

        En este paso se carga el archivo CSV con el
        conjunto de datos que contiene los ejemplos glo-
        up = score_matrix[i-1, j] + gap_penalty
        left = score_matrix[i, j-1] + gap_penalty
        # Se cargan las columnas disponibles y las primeras
        # tres filas para verificación
        score_matrix[i, j] = max(diag, up, left)
    import pandas as pd

# 3. Traceback (Rastreo)
align1, align2 = "", ""
i, j = m, n

nombre_archivo = "cd00001_global_examples.csv"

df = pd.read_csv(nombre_archivo)

while i > 0 and j > 0:
    score = score_matrix[i, j]
    aal, aa2 = seq1[i-1], seq2[j-1]
    match = blosum62.get((aal, aa2), blosum62.get((aa2, aal), -4))
    print(f"Columnas disponibles:", df.columns.tolist())
    #3. filas para revisar, las pares de prote nas
    print("\nPrimeros pares:")
    if score == score_matrix[i-1, j] + match:
        align1 += aal
        align2 += aa2
        i -= 1
        j -= 1
    elif score == score_matrix[i-1, j] + gap_penalty:
        align1 += aal
        align2 += '-'
        i -= 1
    else:
        align1 += '-'
        align2 += aa2
        j -= 1

    print(df.head(3))
    print(df.columns.tolist())

VIII-F. Comparación y Visualización de las
Métricas de Error
Finalmente, se itera sobre la totalidad de los pares
de proteínas del dataset para comparar la precisión
entre el algoritmo clásico Needleman-Wunsch y el
E-Score propuesto. Para optimizar el rendimiento,
el modelo se aloja en la GPU (si está disponible).
Las distancias respecto al alineamiento de referen-
cia son calculadas y posteriormente visualizadas
usando gráficos de barra mediante Matplotlib, ajus-
tando dinámicamente las etiquetas del eje X cuando
la cantidad de datos es muy alta.

import importlib
import numpy as np
import matplotlib.pyplot as plt
import distances_and_extracting_local_alignments as
import torch # Importamos torch para manejar la GPU

importlib.reload(metrics)

return align1[::-1], align2[::-1], score_matrix[m, n]

# 1. Configurar dispositivo (GPU si esta disponible

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Usando dispositivo: {device}") if (idx + 1) % 10 == 0 or idx == 0:
    print(f"Progreso: {Par[idx+1]} completado")
# 2. Asegurarnos de que el modelo de Ankh este en la GPU
model.to(device) print("\nGrafico de comparacion")

alineamientos = [] x = np.arange(len(labels))
errores_nw_dcc = [] width = 0.35
errores_es_dcc = []
labels = [] fig, ax = plt.subplots(figsize=(12, 6))

print("Alineamientos comparativos\n")
rects1 = ax.bar(x - width/2, errores_nw_dcc, width)
rects2 = ax.bar(x + width/2, errores_es_dcc, width)
# 3. Eliminamos .head(5) para analizar TODAS las filas
for idx, row in df.iterrows():
    ax.set_ylabel('Distancia d_cc (Valores mas cercano)')
    ax.set_title('Precision del Alineamiento Biologico')

    seq1 = row['protein1']
    seq2 = row['protein2']
    # Ajuste visual para evitar que el grafico colapse
    if len(labels) <= 20:
        true_a1 = row['ref1']
        true_a2 = row['ref2']
        ax.set_xticks(x)
        ax.set_xticklabels(labels, rotation=45, ha="right")
        ax.bar_label(rects1, fmt='%.3f', padding=3)
        ax.bar_label(rects2, fmt='%.3f', padding=3)

    # Needleman-Wunsch (BLOSUM62)
    nw_a1, nw_a2, nw_score = needleman_wunsch(seq1, seq2, gap_penalty=-11)
    ax.set_xticks([])

    # E-Score (Ankh)
    es_a1, es_a2, es_score = escore.affine_global_alignment_escore_and_traceback(
        seq1=seq1,
        seq2=seq2,
        g_open=-0.25,
        g_ext=-0.01,
        model=model,
        tokenizer=tokenizer
    )
    ax.set_xlabel(f'Pares de proteinas (Total: {len(labels)})')
    ax.legend()
    ax.grid(axis='y', linestyle='--', alpha=0.7)

    fig.tight_layout()
    plt.show()

    reference_alignment = [true_a1, true_a2]
    nw_alignment = [nw_a1, nw_a2]
    es_alignment = [es_a1, es_a2]

    dist_nw = metrics.d_cc(nw_alignment, reference_alignment)
    dist_es = metrics.d_cc(es_alignment, reference_alignment)

    errores_nw_dcc.append(dist_nw)
    errores_es_dcc.append(dist_es)
    labels.append(f"Par {idx+1}")

```